

로지스틱 회귀 — 직선을 확률로 바꾸는 법

이 메일은 스팸일까, 이 환자는 양성일까 — '얼마냐'가 아니라 '어느 쪽이냐'를 맞히는 첫 분류 모델

로지스틱 회귀는 직선 $z = wx + b$ 를 그대로 쓰되, 그 값을 **시그모이드** $\sigma(z) = \frac{1}{1+e^{-z}}$ 에 통과시켜 **0~1 확률**로 바꿉니다. 직선은 $\pm\infty$ 를 오가고 확률은 0과 1 사이에 갇혀 있으니, **승산**과 **로그 승산**을 거쳐 두 범위를 맞춰 줍니다. 학습은 **최대 우도 추정(MLE)**으로 합니다. 관찰된 데이터를 가장 그럴듯하게 설명하는 계수를 경사하강법으로 찾는 방식입니다.

오늘 끝내면 할 수 있는 것

네 가지면 충분합니다.

01

확률로 바꾼다

직선 $wx + b$ 를 왜 그대로 확률로 못 쓰는지 설명하고, **시그모이드**가 그 값을 **0~1 확률**로 바꾸는 과정을 그림으로 풀 수 있다.

02

다리를 건넌다

승산(odds) → 로그 승산이라는 두 칸 다리로 확률의 범위를 **$\pm\infty$ 로 퍼는** 이유를 말하고, 거기서 로지스틱 함수가 나오는 흐름을 잇는다.

03

그럴듯함을 쟀다

확률과 우도를 구분하고, **최대 우도 추정(MLE)**이 '관찰된 데이터를 가장 그럴듯하게 만드는 계수'를 어떻게 경사하강으로 찾는지 안다.

04

쓰고 읽는다

`LogisticRegression`으로 학습하고 `predict_proba`로 확률을 꺼내며, **임계값**을 옮기고 **계수의 부호**로 예측의 이유를 설명할 수 있다.

이 메일은 스팸일까, 아닐까

회귀가 '얼마냐'를 맞혔다면, 분류는 '어느 쪽이냐'를 맞힙니다 — 답이 정해진 몇 개의 보기 중 하나죠.

지도학습의 다른 갈래입니다. 정답이 달린 데이터로 배우는 건 회귀와 똑같지만, 출력이 다릅니다. 집값처럼 **연속된 숫자**면 회귀, 스팸/정상처럼 **정해진 라벨**이면 분류예요.

실무 문제의 상당수가 분류입니다. 메일 필터, 암 진단, 사기 거래 탐지, 이미지 속 동물 맞추기 — "예/아니오" 또는 "A/B/C 중 하나"로 답하는 문제가 모두 분류입니다.

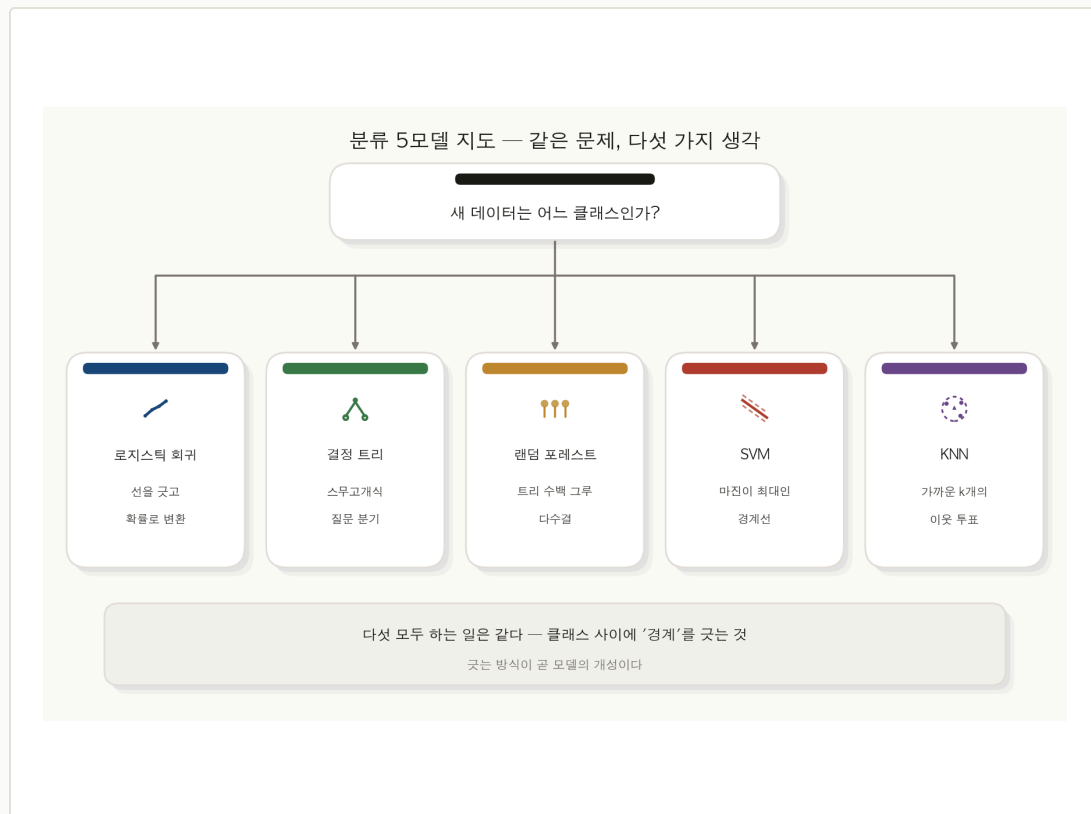
분류 모델은 다섯 식구가 대표적입니다. 로지스틱 회귀·결정 트리·랜덤 포레스트·SVM·KNN — 다섯 모두 하는 일은 같아요. 클래스와 클래스 사이에 **경계를 긋는** 일이죠. 긋는 방식이 저마다의 개성입니다.

그 첫 번째가 로지스틱 회귀입니다. 이름에 '회귀'가 붙어 헷갈리지만 분명한 **분류** 모델입니다. 선형 회귀의 직선 $wx + b$ 를 거의 그대로 가져와, **확률로 바꾸는 한 단계**만 더합니다.

오늘 다룰 순서는 이렇습니다. 왜 직선만으로는 안 되는지 (§2), 직선을 어떻게 확률로 바꾸는지 (§2~3), 그 계수를 어떻게 학습하고 (§3) 어디서 자르는지 (§4) 까지, 한 모델을 깊게 다룹니다.

같은 질문에 다섯 가지 대답 — 분류 모델 지도

분류 모델 다섯이 모두 답하는 질문은 하나예요 — "새 데이터는 어느 클래스인가?". 하는 일은 같습니다. 클래스 사이에 **경계를 긋는** 일이죠. **긋는 방식**이 저마다의 개성이고, 그 중 가장 기본이 되는 것이 로지스틱 회귀입니다.



01 로지스틱 회귀 — 오늘 다룰 모델

직선을 긋고 그 값을 **확률로 변환**합니다. 단순하면서 결과를 숫자로 설명할 수 있는 출발점이에요.

02 결정 트리·랜덤 포레스트

스무고개식 질문으로 데이터를 가르거나(트리), 트리 수백 개루의 다수결로 판단합니다(포레스트). 규칙을 따라가며 가르는 계열이죠.

03 SVM·KNN

두 클래스 사이 여백이 최대인 경계선을 찾거나(SVM), 가까운 이웃 k개의 투표에 맡깁니다(KNN). 같은 문제, 다른 발상이죠.

04 왜 로지스틱부터인가

직선·경사하강법 등 선형 회귀의 도구를 **거의 그대로** 재사용하거든요. 새로 배울 건 '확률로 바꾸는 한 겹'뿐입니다.

딥러닝 시대에도 — 은행과 병원은 이 모델을 쓴다

더 센 모델이 넘쳐나는 2026년에도, **은행의 신용평가**와 **병원의 위험 예측**은 여전히 로지스틱 회귀로 돌아갑니다 — 이유는 성능이 아니라 **신뢰와 설명**에 있어요.

설명할 수 있어야 씁니다. 대출 거절, 암 의심처럼 **근거를 대야 하는** 결정에선 "왜?"에 답 못 하는 모델은 못 씁니다. 로지스틱 회귀는 계수 하나하나가 곧 근거예요.

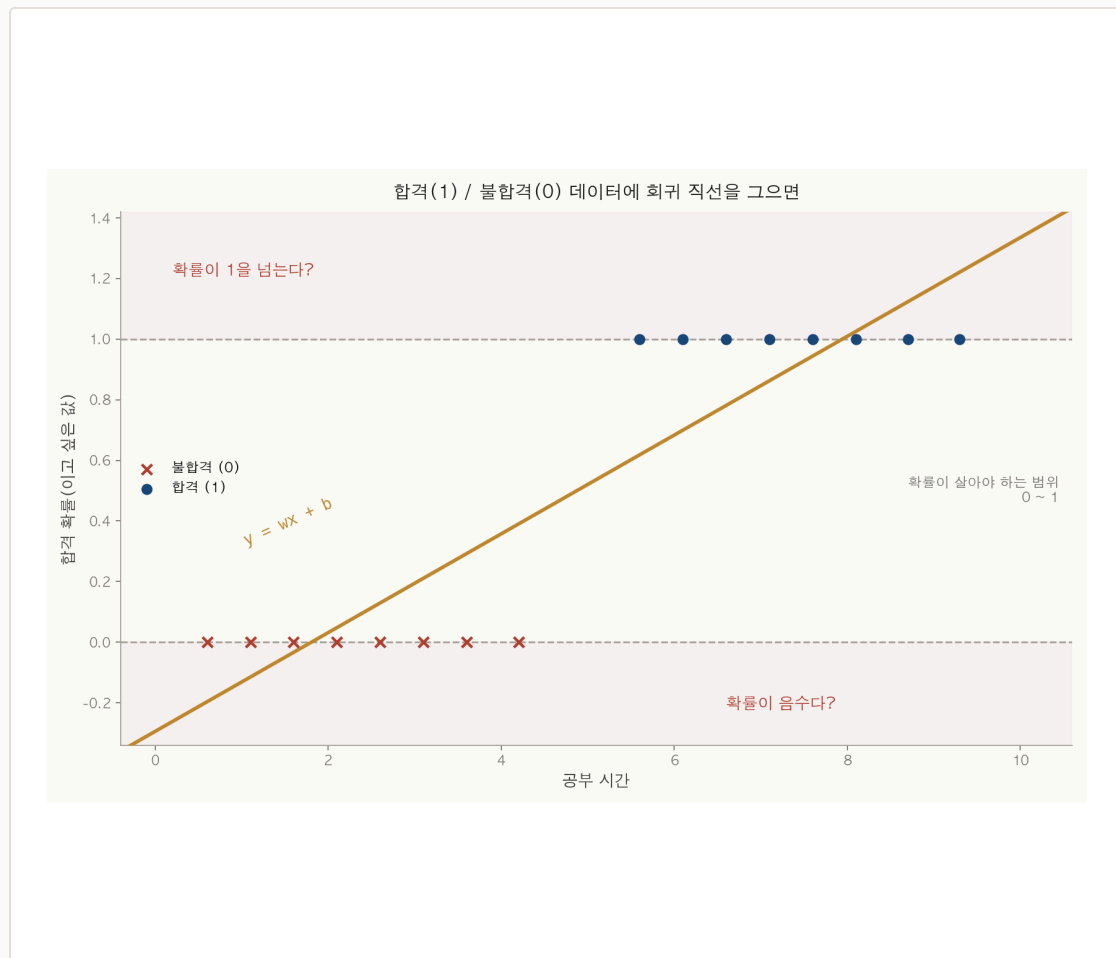
은행 — 신용 스코어카드. 행은 고객, 열은 나이·소득·연체 이력인 평가표에서 부도 확률을 매기는 **스코어카드**는 대부분 로지스틱 회귀입니다. FICO가 1980년대부터 업계 표준이고, **Basel III** 규제가 설명 가능한 모델을 요구하죠.

병원 — 위험 점수표. **Framingham 위험 점수**(2008)는 나이·혈압·콜레스테롤·흡연으로 **10년 내 심혈관 발병 확률**을 매깁니다. 계수의 **승산비(odds ratio)**가 곧 위험 요인의 중요도고요.

규제가 설명을 요구합니다. 2024년 **EU AI Act**는 고위험 AI에 설명가능성을 의무화했고(제13조), 2026년 들어 설명가능 AI(XAI)는 **지켜야 할 요건**으로 굳어집니다. 규제가 셀수록 딥러닝의 불투명함이 걸림돌이죠.

그래서 '먼저' 배웁니다. 작고 빠르고 데이터가 적어도 잘 돌아가, **더 복잡한 모델을 엮기 전의 기준점**이 됩니다. 이 한 모델로 분류 전체의 골격이 잡혀요.

합격/불합격에 직선을 그으면 무슨 일이 날까



y축이 0/1뿐인데 직선은 위로도 아래로도 끝없이 뻗는다 — 확률이 1을 넘고 음수가 된다.

01 정답은 0 아니면 1뿐

공부 시간으로 합격을 맞힌다고 해봅시다. 정답 y 는 **불합격 0, 합격 1** 두 값뿐 — 그 사이 0.5짜리 정답은 없어요. 회귀의 연속값과는 출발부터 다릅니다.

02 직선은 멈출 줄 모른다

$y = wx + b$ 를 그으면 직선은 위아래로 **끝없이** 뻗습니다. 공부를 아주 많이 한 점에선 예측이 **1.4**, 아주 적게 한 점에선 **-0.3**이 나와요.

03 그런데 직선을 버리긴 아깝다

직선엔 큰 장점이 있어요 — **어떤 변수가 얼마나 영향을 주는지**를 계수로 읽고, 그 변수가 통계적으로 유의한지까지 따질 수 있죠. 그래서 식 $w x + b$ 는 그대로 두고 싶습니다. 문제는 오직 **범위**예요.

04 범위만 맞추면 된다

직선은 $\pm\infty$, 확률은 **0~1** — 둘이 안 맞을 뿐입니다. 직선은 그대로 두고, **확률 쪽 범위를 $\pm\infty$ 로 퍼서** 직선과 짝을 맞추기로 해요. 그 다리가 §2.4~2.5의 승산·로그 승산입니다.

범위가 안 맞는다고 식을 버리면, 무엇을 잃는가

직선의 범위가 안 맞는다고 $w x + b$ 를 통째로 버리면, 선형 모델이 가진 **가장 큰 무기**를 함께 버리게 됩니다 — 그 무기는 **해석력**이에요.

계수가 곧 설명입니다. 식 $w x + b$ 에서 각 변수 앞의 계수 w 는 "그 변수가 결과를 어느 방향으로, 얼마나 미치는가"를 숫자 하나로 말해 줍니다. 트리·신경망이 쉽게 내주지 못하는 정보죠.

변수의 유의성을 따질 수 있습니다. 통계적으로 이 변수가 정말 영향을 주는지 (우연이 아닌지)를 검정할 수 있어요. 근거를 문서로 남겨야 하는 **의료·금융** 심사에서 결정적입니다.

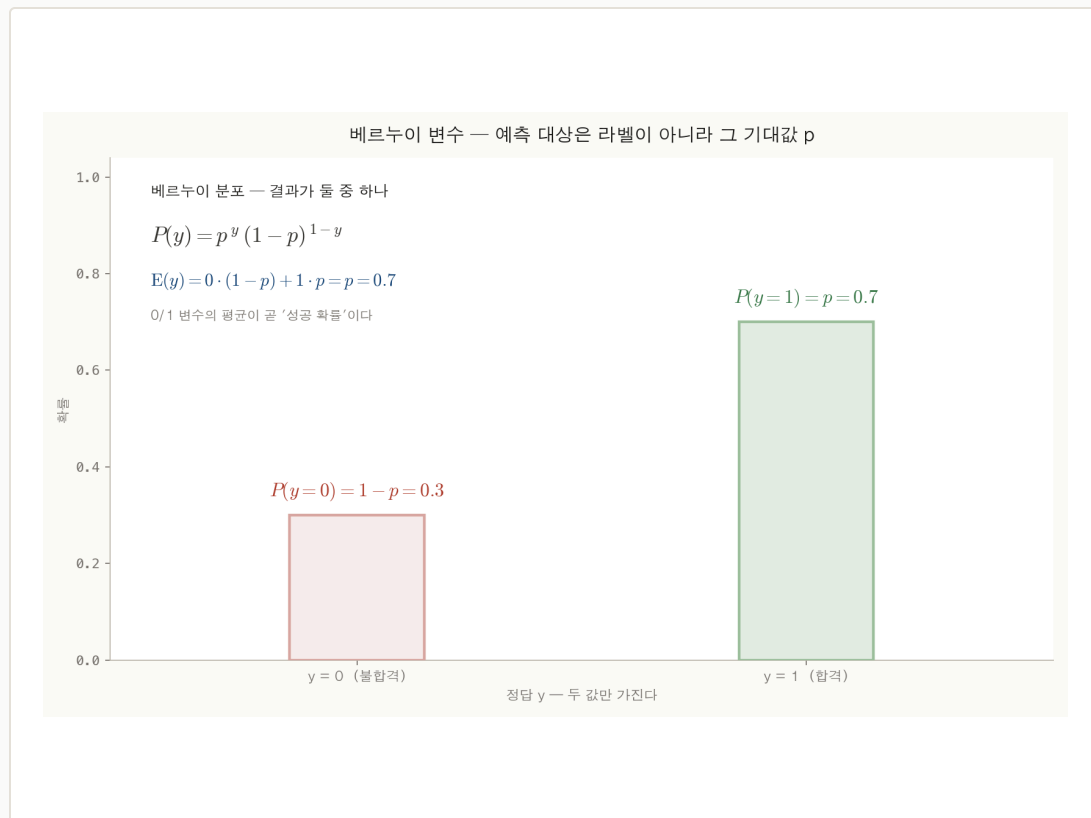
증감의 방향이 분명합니다. "공부 시간이 늘면 합격 확률이 오른다"처럼, 변수가 늘 때 결과가 어느 쪽으로 가는지 **단조롭고 일관되게** 읽힙니다.

학습 도구도 그대로 씁니다. 직선을 유지하면 선형 회귀에서 익힌 **경사하강법**을 거의 그대로 재사용할 수 있어요. 새로 배울 건 '확률로 바꾸는 한 겹'뿐이죠.

그래서 식은 두고, 범위만 고칩니다. 버릴 수 없는 직선은 그대로 두고, 대신 **확률 쪽 범위**를 $\pm\infty$ 로 퍼 둘을 맞춥니다 — §2.4부터 그 다리를 한 칸씩 놓습니다.

우리가 진짜 예측하는 건 라벨이 아니라 '성공 확률 p'

정답 y 는 **0 아니면 1** 두 칸뿐 — 동전 앞/뒤 같은 **베르누이 변수**입니다. 이 변수는 한 줄로 적혀요: $P(y) = p^y(1-p)^{1-y}$. 그 **기대값(평균)**은 $E(y) = p$, 곧 **1이 나올 확률**이죠. 그래서 모델이 내놓을 값은 라벨이 아니라 그 뒤의 **확률 p**이고, 남은 일은 이 **p를 직선 $w x + b$ 로 잇는 것**뿐입니다.



01 값이 두 개뿐인 변수

베르누이 변수는 결과가 **둘 중 하나**. 합격/불합격, 스팸/정상처럼 분류의 정답이 모두 이 모양이에요.

02 한 식으로 묶는다

$P(y) = p^y(1-p)^{1-y}$. 지수 자리에 $y = 1$ 을 넣으면 p , $y = 0$ 을 넣으면 $1 - p$ — 두 경우가 **한 줄**에 담깁니다. 이 한 줄이 §3 우도의 **재료**가 돼요.

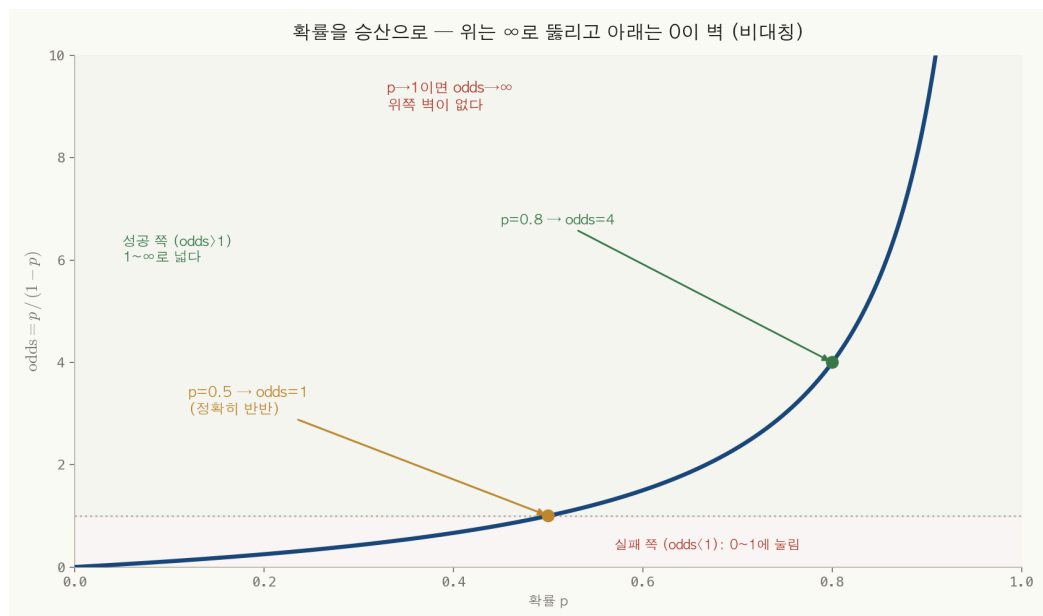
03 기대값이 곧 확률

$E(y) = 0 \cdot (1 - p) + 1 \cdot p = p$. **0/1 변수의 평균은 성공 확률 그 자체**라는 게 핵심입니다(주사위 기대값이 3.5인 것과 같은 계산이죠).

04 그래서 목표는 p

라벨은 예측의 **마지막 한 걸음**일 뿐, 모델의 본체는 **확률 p를 잘 맞추는 것**. 좌변 p 는 0~1, 우변 $w x + b$ 는 $\pm\infty$ — §2.4~2.5에서 범위를 펴 둘을 잇습니다.

승산(Odds) — 확률을 '성공 대 실패' 비율로 바꾼다



01 성공 대 실패의 비율

성공 확률이 p 일 때, 실패 대비 성공의 비율이 **승산**입니다: $\text{odds} = \frac{p}{1-p}$. $p = 0.8$ 이면 $0.8/0.2 = 4$ — 성공이 실패보다 **4배** 잘 일어난다는 뜻, 도박판의 "4:1"이죠.

02 양 끝에서 무슨 일이

$p = 1$ (무조건 성공)이면 분모가 0이라 odds는 **무한대로** 치솟고, $p = 0$ (무조건 실패)이면 odds는 **0**. 그 사이 odds = 1이 정확히 반반($p = 0.5$)의 자리예요.

03 범위가 0 ~ ∞로 퍼진다

확률의 위쪽 벽(1)이 사라졌습니다. p 가 1에 다가갈수록 odds는 끝없이 커져요 — 확률 (0~1)보다 **한 칸 넓은** 0~∞가 됐습니다.

04 그래도 아직 비대칭

위(1~∞)는 넓고 아래(0~1)는 좁아요. 아래로는 여전히 **0이 벽** — 직선의 $\pm\infty$ 대칭과는 안 맞습니다. 그래서 한 칸 더, **로그**가 필요해요 (§2.5).

승산 곡선 — p 가 1에 다가갈수록 ∞로 치솟고, 아래는 0이 벽. 1을 기준으로 위아래가 비대칭이다.

log 한 번이면 양변의 범위가 같아진다

직선 $w x + b$ 는 $\pm\infty$ 를 오가는데 확률 p 는 **0~1**에 갇혀 있어요 — 이 **범위 불일치**가 핵심 문제입니다. §2.4의 승산($0 \sim \infty$, 비대칭)에 **로그**를 한 번 씌우면 **로그 승산**($-\infty \sim +\infty$)이 되어, 오른쪽 곡선처럼 **0을 기준으로 대칭**이 되며 직선과 범위가 맞습니다.



01 1칸 — 비율로(odds)

확률을 $\frac{p}{1-p}$ 로 바꿔 위쪽 벽을 허뭅니다. $0 \sim \infty$ 로 퍼지지만 **아래는 여전히 0이 벽** — 비대칭이 남아요.

02 2칸 — log로(log-odds)

거기에 로그를 씌우면 $\log \frac{p}{1-p}$ 가 $-\infty \sim +\infty$. odds가 1보다 작으면 음수, 크면 양수 — **0을 기준으로 대칭**이 됩니다.

03 이게 로지스틱 회귀의 모델 식

$\log \frac{p}{1-p} = w x + b$ — 양변 모두 $\pm\infty$ 라 **드디어 등호가 성립**합니다. 좌변을 **로짓(logit)**이라 불러요. 직선이 확률의 '로그 승산'을 직접 만드는 셈이죠.

04 다음은 거꾸로 풀기

쓸 때는 p 가 필요하니 이 식을 **p 에 대해 풀어** 봅니다. 그러면 우리가 안 만든 **시그모이드가 저절로 튀어나와요** — 다음 장에서 식으로 한 단계씩 유도합니다.

로짓을 p에 대해 풀기 — exp 한 번으로 시그모이드까지

로짓을 p에 대해 풀기 — exp 한 번으로 시그모이드까지

$$\log \frac{p}{1-p} = z$$

출발 — 모델 식 (로짓 승산 = z)



$$\frac{p}{1-p} = e^z$$

① 양변에 exp 를 씌운다 — 로그가 사라진다



$$p = e^z(1-p) = e^z - pe^z$$

② (1-p) 를 곱해 분수를 푼다



$$p(1 + e^z) = e^z$$

③ p 가 든 항을 한쪽으로 모은다



$$p = \frac{e^z}{1 + e^z} = \frac{1}{1 + e^{-z}} = \sigma(z)$$

④ (1 + e^z) 로 나눈다 — 시그모이드!

01 ① exp로 로그를 벗긴다

§2.5의 $\log \frac{p}{1-p} = z$ 양변에 **지수(exp)**를 씌우면 로그가 사라지고 $\frac{p}{1-p} = e^z$ —

승산이 e^z 라는 깔끔한 식이 됩니다.

02 ② 분수를 푼다

양변에 (1 - p)를 곱하면 $p = e^z(1-p) = e^z - pe^z$. 분수가 사라지고 p가 식 양쪽에 흩어집니다.

03 ③ p를 한쪽으로 모은다

p가 든 항을 모아 공통인수로 묶으면 $p(1 + e^z) = e^z$. 중학교에서 본 그 인수분해 정리예요.

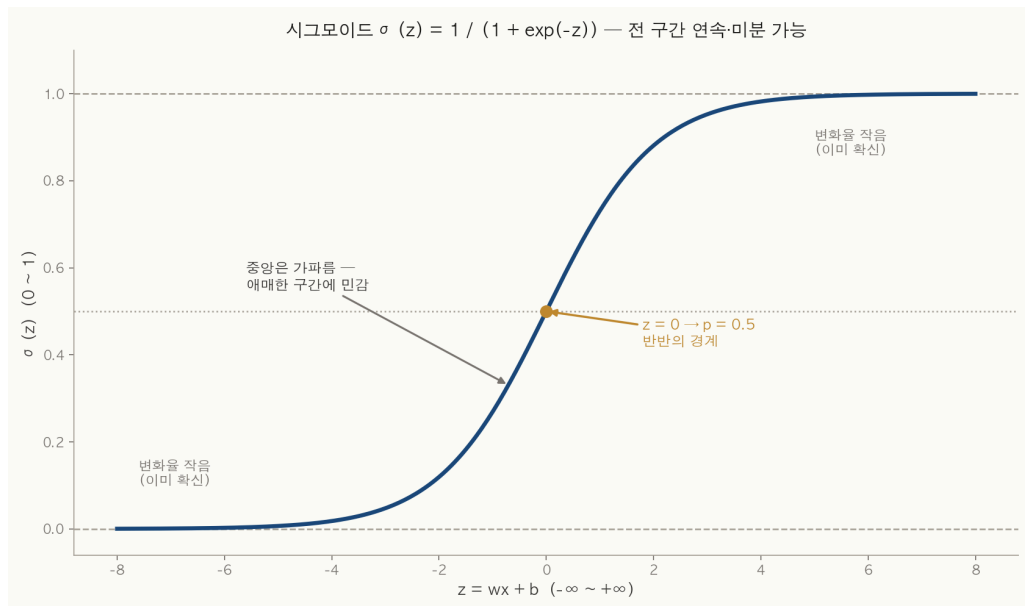
04 ④ 나누면 시그모이드

(1 + e^z)로 나누면 $p = \frac{e^z}{1 + e^z} = \frac{1}{1 + e^{-z}} = \sigma(z)$. 따로 가져온 함수가 아니라 **로짓에 exp를 씌워 푼 결과**가 곧 시그모이드입니다.

로짓 승산 = z 에 exp를 씌우고 정리하면, 따로 가져오지 않은 시그모이드 $\sigma(z) = \frac{1}{1+e^{-z}}$ 가 나온다.

유도로 얻은 그 S자 곡선 — 어떤 함수인가

방금 유도한 **시그모이드** $\sigma(z) = \frac{1}{1+e^{-z}}$ 는 어떤 실수 z 가 들어와도 **0~1 사이**로 변환해 내보내는 S자 함수입니다. 직선의 출력 $z = wx + b$ 를 통과시키면 비로소 **확률**로 읽을 값이 나와요. 왜 이 함수가 분류에 잘 맞는지, 모양을 뜯어봅시다.



01 양 끝은 납작, 중앙은 가파름

z 가 크면 1에, 작으면 0에 **바짝 붙어** 절대 넘지 않습니다. 0~1이라는 확률의 방을 벗어나지 못하는 거죠.

02 $z = 0$ 이 반반의 경계

$\sigma(0) = 0.5$ — 직선 값이 0인 지점이 합격·불합격 **5:5**입니다. 여기서 멀어질수록 한쪽으로 확신이 쏠려요.

03 중앙이 민감한 게 장점

경계 근처(애매한 구간)에선 작은 z 변화가 확률을 크게 흔듭니다. 정작 **헛갈리는 곳에서 예민하게** 반응하는 셈이에요.

04 전 구간 미분 가능

끊긴 데 없이 매끄러워 **경사하강법**과 궁합이 좋습니다 — 선형 회귀의 그 학습 루프를 그대로 쓸 수 있는 이유예요.

동전 10번에 앞면 7번 — 확률이나, 우도나

여기까지 모델의 **모양**은 끝 — 그런데 계수 w, b 는 아직 미정입니다. 데이터로 그 값을 정하는 게 **학습**이고, 그 채점표가 바로 **우도**예요. 헷갈리기 쉬운 **확률**과 먼저 가릅니다 — 동전 하나로요.

모델을 알고, 데이터를 묻는다

확률 Probability

파라미터가 먼저다

"이 동전은 공정하다($p=0.5$)"처럼 **모델을 정해 놓고**, 거기서 어떤 데이터가 나올 가능성을 따집니다.

이론에서 출발

"공정한 동전을 던지면 앞면이 나올 확률은 50%" — 데이터를 보기 **전에** 이론으로 계산하는 값이에요.

방향: 모델 → 데이터

파라미터를 고정하고 결과의 가능성을 잹니다. 우리가 학교에서 배운 '확률'이 보통 이쪽이죠.

데이터를 알고, 모델을 묻는다

우도 Likelihood

데이터가 먼저다

"10번 던져 7번 앞면이 **나왔다**" — 관찰을 고정해 놓고, 어떤 파라미터가 이 결과를 만들었는지 거꾸로 묻습니다.

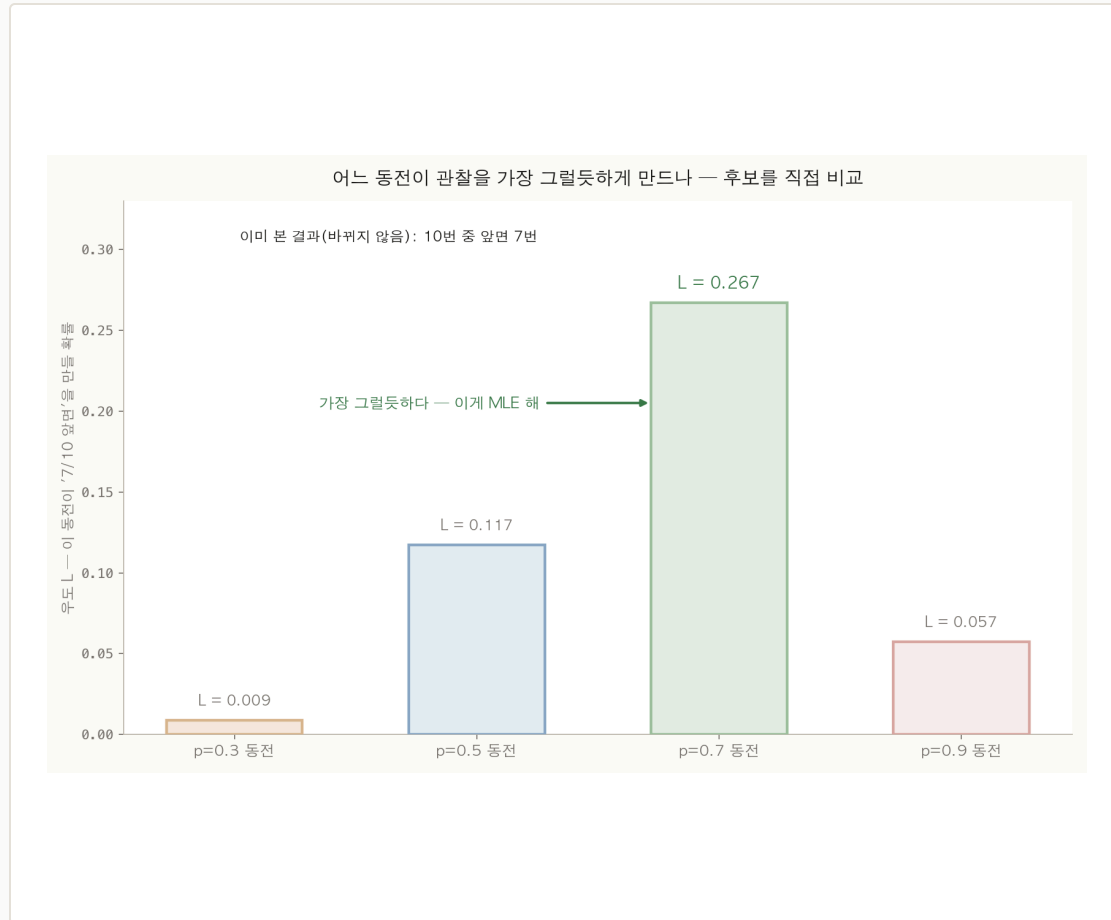
경험에서 출발

"이 데이터라면 $p=0.7$ 일 가능성은 얼마인가?" 같은 질문이에요. 관찰된 사실에 비춰 **모델을 평가**합니다.

방향: 데이터 → 모델

데이터를 고정하고 파라미터의 그럴듯함을 잹니다. **학습**이 바로 이 방향 — 데이터로 계수를 찾는 일이에요.

동전 네 개로 골라보기 — 어느 게 가장 그럴듯한가



이미 본 결과(10번 중 7번 앞면)는 그대로 두고, 동전 후보만 바꿔 우도를 재면 $p=0.7$ 이 가장 크다 — 그게 MLE 해다.

01 이미 본 결과는 못 바꾼다

동전을 10번 던졌더니 **앞면이 7번 나왔다** — 이걸 **이미 일어난 사실**이라 바뀌지 않아요. 모르는 건 딱 하나, '이게 **어떤 동전**(앞면 확률 p)이었나'입니다.

02 그래서 동전만 바꿔 끼운다

사실(7/10)은 그대로 두고, 동전 후보를 $p=0.3 \cdot 0.5 \cdot 0.7 \cdot 0.9$ 로 **바꿔 끼우며** 묻습니다 — "이 동전이었다면 7/10이 나올 만했나?"

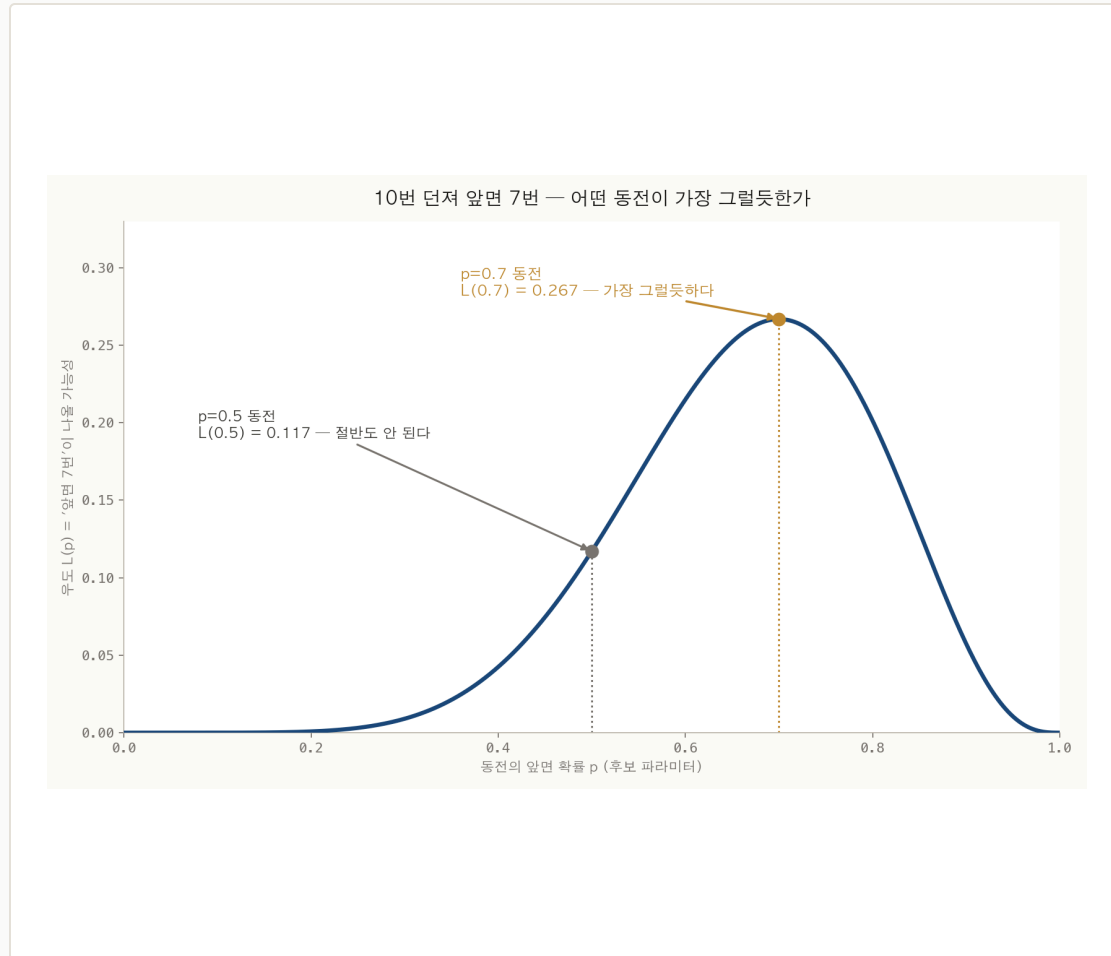
03 가장 그럴듯한 동전이 답

$p=0.3$ 동전이라면 0.009(거의 안 나옴), $p=0.7$ 동전이라면 **0.267**로 가장 큼니다. 본 결과를 **가장 그럴듯하게** 설명하는 $p=0.7$ 이 답 — 이게 **최대 우도 추정(MLE)**이에요.

04 로지스틱도 똑같다

동전의 p 자리에 **직선의 계수** w, b 가 들어갈 뿐이에요. 합격/불합격 기록을 가장 그럴듯하게 만드는 w, b 를 고르는 게 로지스틱의 학습입니다.

후보가 무수히 많으면 — 우도 곡선의 꼭대기가 MLE



01 네 개가 아니라 전부

§3.2에선 동전 네 개만 비교했지만, 후보 p 는 0과 1 사이에 **무한히** 많습니다. 전부 훑어 우도를 이으면 **연속 곡선**이 돼요.

02 우도 곡선 = 언덕 모양

각 p 의 우도 $L(p) = \binom{10}{7} p^7 (1-p)^3$ 를 세로로 찍으면 가운데가 솟은 **언덕**입니다.
 $L(0.5) = 0.117$, $L(0.7) = 0.267$.

03 꼭대기가 MLE 해

그 봉우리의 **정상**에 있는 p 가 데이터를 가장 잘 설명하는 답이에요. 기울기가 0이 되는 그 지점을 찾는 게 §3.4의 경사 상승입니다.

04 계수가 여럿이면 산맥

변수가 둘 이상이면 봉우리는 **여러 차원의 산**이 됩니다. 손으로 못 찾으니 경사하강(상승)으로 한 걸음씩 정상을 향해 올라가요.

후보 p 를 0~1 전부 훑어 우도를 찍으면 봉우리가 생긴다 — 그 꼭대기($p=0.7$)가 MLE 해다.

우도를 최대로 — 로지스틱 회귀가 배우는 다섯 걸음

회귀의 학습 루프와 뼈대가 같습니다. 채점표가 MSE에서 **로그 우도**로 바뀐 것뿐이에요.

STEP 01

초기화

계수 w, b 를 임의의 값으로 둡니다.
처음 그은 직선은 형편없어도
괜찮아요 — 출발점일 뿐이니까.



STEP 02

우도 계산

지금 계수로 각 데이터의 예측 확률을
구하고, **관찰된 라벨이 나올 가능성**을
전부 곱해 우도를 만듭니다.



STEP 03

로그 우도

곱은 다루기 어려워 **로그**를 씌워
합으로 바꿉니다 — $\log L =$
 $\sum [y \log p + (1 - y) \log(1 - p)]$.
작은 확률을 곱하다 0으로 새는
사고도 막아줘요.



STEP 04

경사 상승

로그 우도를 **높이는** 방향으로 w, b 를
한 걸음씩 고칩니다 — 회귀의
경사하강을 거울로 뒤집은 셈이에요.



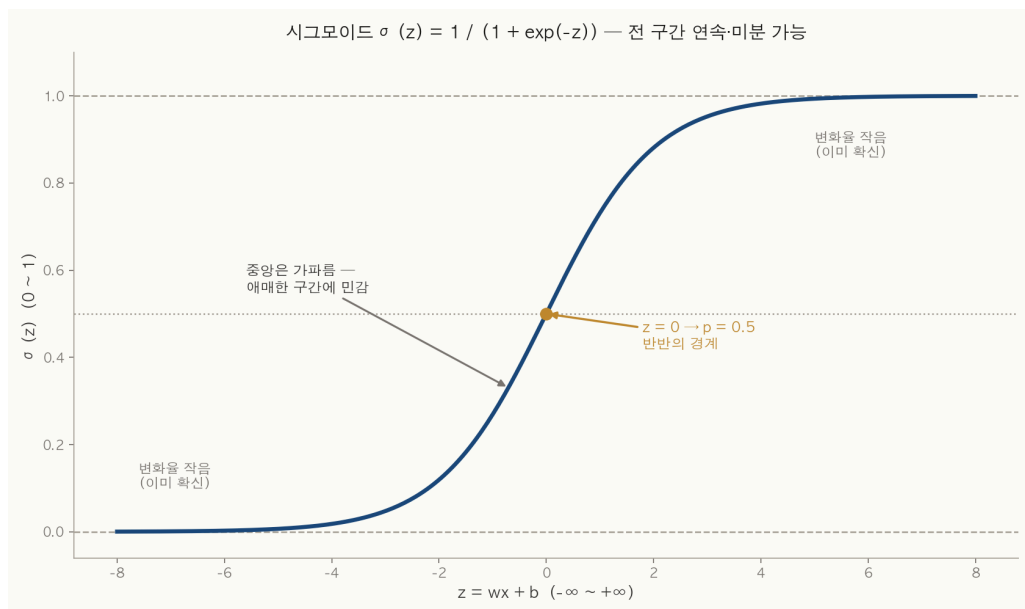
STEP 05

수렴

기울기가 거의 0이 되면 우도가 더는
안 오릅니다. 그 자리의 계수가 **최종 답**
— 학습 끝입니다.

공부 3시간이면 합격 확률은? — z 를 직접 굴려 본다

초기 계수 $w = 1$, $b = -2$ 로, **공부 시간 3시간**인 학생의 합격 확률을 손으로 구해봅시다. ① 직선으로 z 를 만들고 ② 시그모이드에 넣으면 끝 — 모델이 한 샘플을 예측하는 **전 과정**이 두 줄입니다.



01 ① 직선 — $z = wx + b$

$z = 1 \cdot 3 + (-2) = 1$. 공부 시간 3을 직선에 넣어 **로짓 값 1**을 얻습니다. 여기까진 선형 회귀와 똑같아요.

02 ② 시그모이드 — $p = \sigma(z)$

$\sigma(1) = \frac{1}{1+e^{-1}} = 0.73$. $z = 1$ 을 0~1 사이로 변환하니 **합격 확률 약 73%**가 나옵니다.

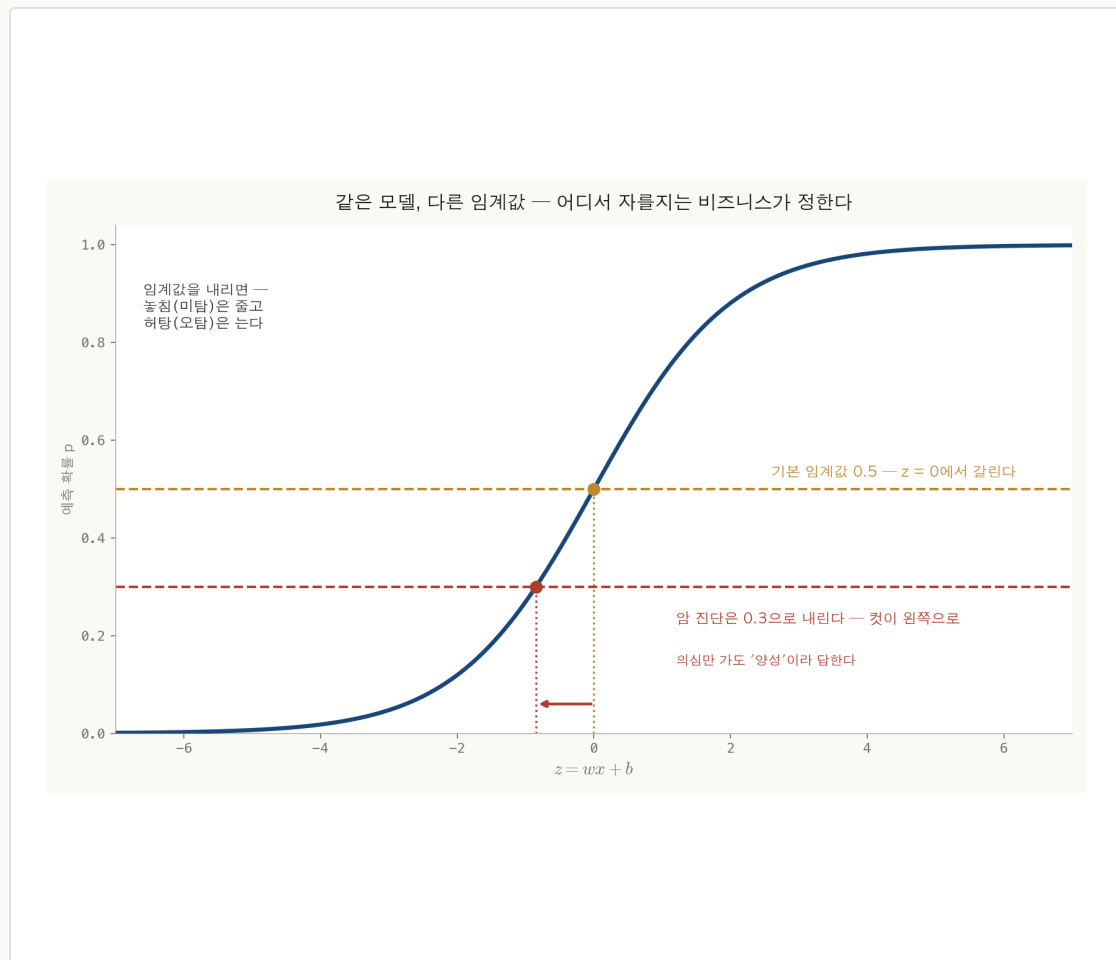
03 ③ 라벨 — 0.5와 비교

$0.73 > 0.5$ 이므로 예측은 **합격(1)**. z 가 0보다 컸으니(σ 가 0.5 위) 당연한 결과죠.

04 학습은 이 w, b 를 고치는 일

지금 $w = 1$, $b = -2$ 는 초기값일 뿐입니다. MLE가 데이터를 보며 이 두 숫자를 **더 그럴듯한 값**으로 다듬어 가요.

확률을 라벨로 — 어디서 자를지는 누가 정하나



기본 임계값 0.5($z=0$)에서 자르지만, 컷라인은 비용에 따라 옮길 수 있다.

01 모델의 출력은 확률이다

로지스틱 회귀가 내놓는 건 0.73 같은 **확률**이지 0/1 라벨이 아닙니다. 라벨로 바꾸려면 "몇 % 이상이면 합격"이라는 **기준선**이 하나 필요해요.

02 기본 임계값은 0.5

특별한 이유가 없으면 $p \geq 0.5$ 를 1로, 미만을 0으로 봅니다. 시그모이드에서 0.5는 $z = 0$ 이라, 결정 경계는 직선 $w x + b = 0$ 이 돼요.

03 암 진단은 0.3으로 내린다

놓치면 큰일 나는 문제에선 임계값을 낮춥니다. $p \geq 0.3$ 이면 양성 — **의심만 가도 정밀검사**로 보내는 거죠. 놓침(미탐)은 줄고 허탕(오탐)은 늘어요.

04 컷은 비즈니스가 정한다

오탐과 미탐 중 **어느 쪽이 더 비싼지**에 따라 임계값이 갈립니다. 같은 모델이라도 스팸 필터와 암 진단은 자르는 자리가 달라요 — 모델 밖의 결정입니다.

보기가 셋 이상이면 — 소프트맥스로 확장한다

지금까지는 합격/불합격, **둘 중 하나**였습니다. 그럼 "고양이/개/새"처럼 셋 이상이면 어떻게 할까요?

score

클래스마다 직선 점수 하나

이진 분류는 직선이 하나였지만, 다중 분류는 **클래스마다 자기 직선**을 둡니다. 고양이 점수·개 점수·새 점수 — 입력 하나에 점수가 클래스 수만큼 나와요.

softmax

점수 여러 개를 확률로

시그모이드의 다중 버전 $\text{softmax}(z_k) = \frac{e^{z_k}}{\sum_j e^{z_j}}$ 입니다. 여러 점수를 받아 **모두 0~1이고 합이 1**이 되도록 변환해요 — "고양이 0.7, 개 0.2, 새 0.1"처럼 **클래스별 확률 분포**가 나옵니다.

argmax

가장 높은 확률을 고른다

마지막은 간단합니다 — 확률이 가장 큰 클래스를 답으로 내놓아요. sklearn에선 옵션 하나로 자동 처리되니, **발상만 알면** 충분합니다.

예측의 이유를 말할 수 있다 — 계수 읽는 법



계수 w 는 그 변수가 로그 승산을 미치는 방향과 세기 — 부호와 크기에 의미가 담긴다.

01 부호 = 미는 방향

계수가 **양수면** 그 변수가 클수록 합격 확률이 올라가고, **음수면** 내려갑니다. 공부 시간 +1.2, 결석 횟수 -0.8 — 상식과 부호가 맞아요.

02 크기 = 미는 세기

절댓값이 클수록 영향이 셉니다. $|+1.2| > |-0.8|$ 이면 **공부 한 시간이 결석 한 번보다** 합격에 더 크게 작용한다는 뜻이에요.

03 정확히는 '로그 승산'을 민다

w 는 직선이 만드는 **log-odds**에 더해집니다. e^w 로 바꾸면 "공부 1시간당 합격 승산 $e^{1.2} \approx 3.3$ 배"처럼 **승산 배수**로 읽을 수도 있죠.

04 이 해석 가능성이 장점이다

트리·신경망과 달리 "왜 이렇게 예측했는지"를 **변수 단위로 설명**할 수 있습니다. 대출 심사·의료처럼 **근거를 대야 하는** 곳에서 로지스틱 회귀를 자주 쓰는 이유입니다.

sklearn으로는 세 줄 — fit, predict, 그리고 확률

구조는 회귀와 똑같습니다 — `fit() → predict()`. 클래스 이름만 `LogisticRegression`으로 바뀌었죠. 달라지는 건 출력의 성격입니다. 이 모델은 라벨뿐 아니라 **확률**과 **계수**를 함께 내놓아요.

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# X: 공부 시간·결석 횟수 등 단서 / y: 합격(1) / 불합격(0)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# C = 1/규제강도 (작을수록 강한 L2 규제) — 기본 L2가 켜져 있다
model = LogisticRegression(C=1.0, max_iter=1000)
model.fit(X_train, y_train)          # 로그 우도를 최대 — 경사하강

pred = model.predict(X_test)         # 임계값 0.5로 0/1 라벨
print(accuracy_score(y_test, pred)) # 0.91 — 맞힌 비율
```

01 같은 fit / predict

회귀에서 익힌 인터페이스 그대로입니다. `fit` 안에서 §3의 **로그 우도 최대화**가 벌어지고, `predict`는 임계값 0.5로 0/1 라벨을 내줘요.

02 C — 규제 손잡이

로지스틱 회귀는 **L2 규제가 기본**으로 켜져 있습니다. `C`는 규제 강도의 **역수** — 작을수록 강하게 규제해(계수를 누름) 과적합을 막아요. 회귀의 `alpha`와 방향이 반대인 셈이죠.

03 accuracy로 채점

분류의 첫 채점표는 **맞힌 비율(정확도)**입니다. 다만 정확도만으론 부족한 경우가 많아 — 정밀도·재현율 같은 지표는 뒤에서 따로 다뤄요.

04 max_iter — 경사하강 횟수

우도를 최대로 만드는 반복 횟수의 상한입니다. 수렴 경고가 뜨면 이 값을 키워, 경사가 **충분히 바닥까지** 내려가게 해줘요.

이름은 닮았지만 — 선형 회귀 vs 로지스틱 회귀

둘 다 직선 $w x + b$ 에서 출발합니다. **그 값을 어떻게 쓰느냐**가 달라요.

숫자를 그대로 내놓는다

선형 회귀 Regression

출력 = 연속값

직선의 값 $w x + b$ 를 **그대로** 예측으로 씁니다. 집값 4.23억처럼 실수 축 어디든 떨어질 수 있어요.

채점 = MSE

예측과 실제의 **오차 제곱**을 줄입니다. 작을수록 좋은 선, 경사하강법으로 바닥을 찾죠.

푸는 문제 = 회귀

"얼마나"를 맞히는 문제 — 기온·매출·시세처럼 **연속된 수치** 예측에 씁니다.

확률로 바꿔 라벨을 고른다

로지스틱 회귀 Classification

출력 = 확률 → 라벨

직선 값을 **시그모이드**에 통과시켜 0~1 확률로 만든 뒤, 임계값으로 0/1을 고릅니다.

채점 = 로그 우도

MSE 대신 **우도를 최대화**(=로그 손실 최소화)합니다. 학습 루프의 뼈대는 그대로예요.

푸는 문제 = 분류

"어느 쪽이냐"를 맞히는 문제 — 스팸·진단·사기처럼 **정해진 라벨** 분류에 씁니다.

predict_proba — 라벨 뒤에 숨은 확신도를 꺼낸다

분류의 진짜 출력은 라벨이 아니라 **확률**입니다. `predict_proba` 로 확신도를 직접 보면, 임계값을 옮기고(§4.1) 계수로 이유를 설명하는(§4.3) 일이 전부 코드 몇 줄로 이어져요.

```
# 라벨이 아니라 '확률'을 직접 본다 — 분류의 진짜 출력
proba = model.predict_proba(X_test) # 각 행: [P(0), P(1)]
print(proba[:3])
# [[0.12 0.88]      → 클래스 1일 확률 0.88
#   [0.73 0.27]      → 클래스 1일 확률 0.27
#   [0.41 0.59]]      → 0.5 턱걸이 — 애매한 샘플

# 임계값을 직접 옮긴다 — predict는 0.5 고정이지만
high_recall = (proba[:, 1] >= 0.3).astype(int) # 암 진단 모드

# 계수가 곧 영향력 보고서 — 부호=방향, 크기=세기
print(model.coef_)      # [[ 1.20 -0.80 ]] 공부(+) 결석(-)
print(model.intercept_) # [-2.0] = b
```

01 predict_proba — 확률을 직접

각 샘플마다 `[P(0), P(1)]` 를 돌려줍니다. 0.88처럼 확신하는 예측과 0.59처럼 **턱걸이** 예측을 구분할 수 있어, 위험한 결정의 경계를 알게 돼요.

02 임계값은 직접 옮긴다

`predict` 는 0.5 고정이지만, `proba[:, 1] >= 0.3` 처럼 **직접 자르면** 임계값을 내릴 수 있습니다 — §4.1의 암 진단 모드를 코드로 옮긴 거죠.

03 coef_ — 영향력 보고서

학습된 계수 배열입니다. 부호로 방향을, 크기로 세기를 읽어오 — §4.3에서 본 +1.2(공부)--0.8(결석)이 여기 그대로 담깁니다.

04 intercept_ = b

절편 b가 따로 담깁니다. 손계산의 $b = -2$ 처럼, 모든 변수가 0일 때의 기본 로그 승산을 정하는 값이에요.

정리 — 직선에 확률을 더한 첫 분류 모델

오늘 한 일

- ▶ **분류** — '얼마냐'가 아닌 '어느 쪽이냐', 라벨을 맞히는 지도학습
- ▶ **직선 → 확률** — $w x + b$ 를 시그모이드로 바꿔 0~1로
- ▶ **승산 → 로그 승산** — 범위를 $\pm\infty$ 로 퍼는 두 칸 다리, 로짓
- ▶ **MLE** — 관찰을 가장 그럴듯하게 만드는 계수를 경사로 찾는다
- ▶ **임계값·계수** — 0.5에서 자르고(웁길 수 있고), 부호로 이유를 읽는다

직선은 $\pm\infty$ 를, 확률은 0~1을 오간다. 그래서 승산과 로그로 범위를 맞춰 둘을 잇고, 시그모이드로 확률을 읽는다. 학습은 관찰을 가장 그럴듯하게 만드는 계수를 찾고(MLE), 예측은 확률을 임계값으로 자른다. 이렇게 회귀가 분류로 확장된다.

더 나아가려면

- ▶ **직접 돌려보기** — `LogisticRegression`으로 학습, `predict_proba`로 확률 확인
- ▶ **계수 읽기** — 부호로 각 특징이 확률을 올리는지 내리는지 해석
- ▶ **임계값 조정** — 0.5를 옮겨 가며 정밀도·재현율의 균형 맞추기
- ▶ **평가 넓히기** — 정확도뿐 아니라 정밀도·재현율·ROC도 함께
- ▶ **다른 분류 모델** — 트리·랜덤 포레스트·SVM·KNN과 비교해 보기